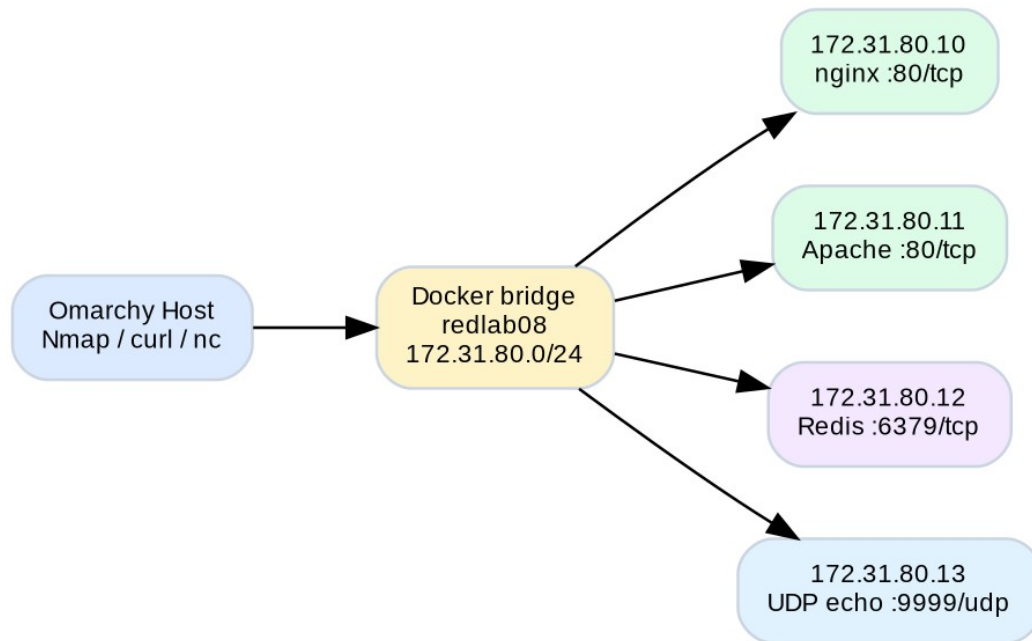


## RED TEAM LEARNING PATH

## MODULE 08

## Network Scanning &amp; Enumeration

Omarchy / Arch Linux Edition

*Training topology used throughout this module.***LEGAL & SAFETY BOUNDARY**

Semua scan pada hands-on module ini dibatasi ke loopback dan subnet Docker 172.31.80.0/24 yang dibuat sendiri. Jangan pindahkan command mentah ke jaringan publik, kantor, kampus, Wi-Fi tetangga, cloud account, atau target lain tanpa authorization tertulis dan scope yang jelas.

Version 1.0 | 04 Oktober 2026

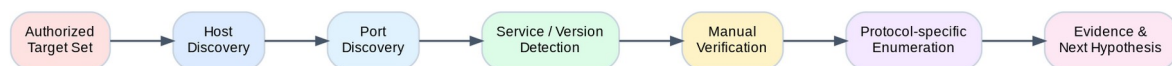
| Field        | Value                                                                                                        |
|--------------|--------------------------------------------------------------------------------------------------------------|
| Prerequisite | Module 00–07, khususnya networking, HTTP, Linux, scope/RoE, recon                                            |
| Core tools   | nmap, curl, openbsd-netcat, jq, Python 3, Docker                                                             |
| Lab subnet   | 172.31.80.0/24 (Docker bridge, tidak dipublish ke host/LAN)                                                  |
| Outcome      | Mengubah daftar target authorized menjadi inventory host/port/service + bukti manual yang dapat diverifikasi |

```
omarchy pkg add nmap curl openbsd-netcat jq python
```

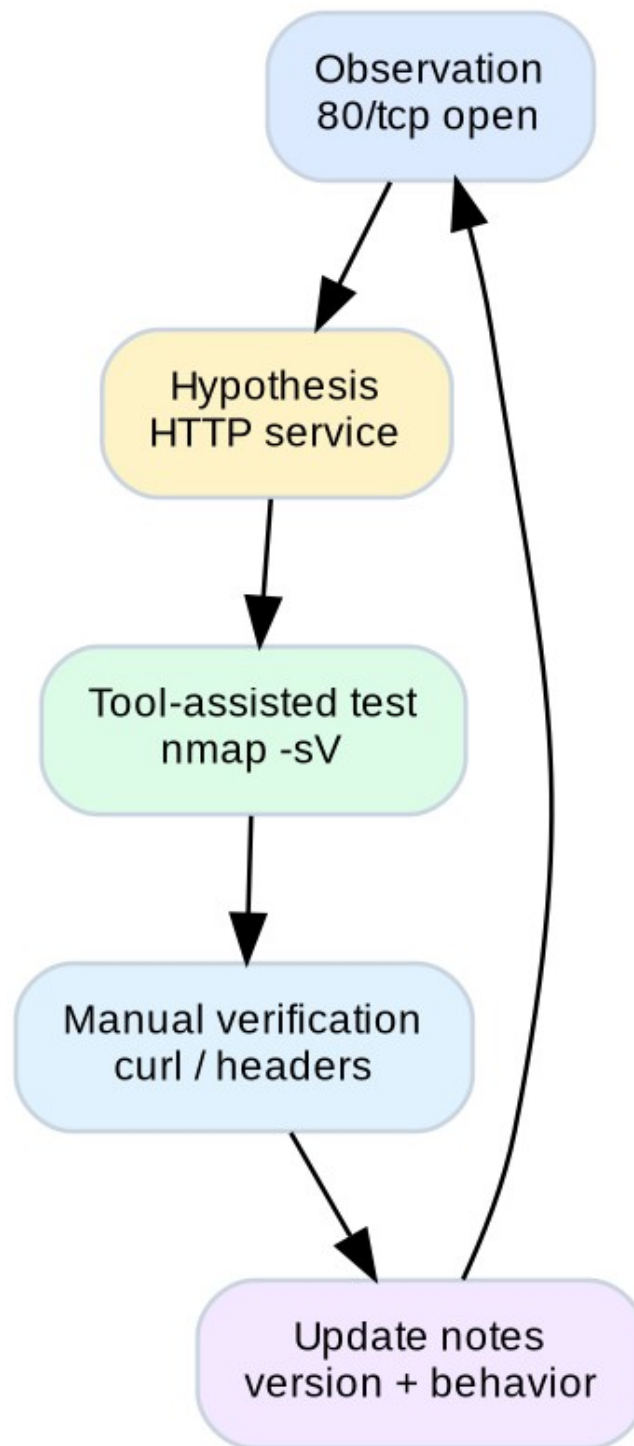
## 1. Learning Objectives

- [ ] Membedakan **host discovery**, **port discovery**, **service detection**, dan **protocol-specific enumeration** sebagai empat tahap berbeda.
- [ ] Menjelaskan enam state port Nmap dan kenapa state itu bergantung pada sudut pandang scanner — bukan sifat permanen port.
- [ ] Memilih antara TCP connect scan, SYN scan, dan UDP scan sesuai konteks (privilege, kecepatan, akurasi).
- [ ] Menggunakan `-sv`, output XML, dan NSE secara eksplisit dan hati-hati.
- [ ] Melakukan manual verification agar hasil scanner tidak diperlakukan sebagai kebenaran absolut.
- [ ] Menyimpan hasil scan sebagai evidence dan mengubah XML menjadi inventory CSV terstruktur.
- [ ] Melakukan scanning hanya pada target yang sudah masuk scope — guardrail teknis bukan pengganti authorization.

## 2. Mental Model: Enumeration Adalah Loop, Bukan Satu Command



*Enumeration pipeline: discovery sampai evidence dan next hypothesis.*



*Tool membantu pengujian; manual verification menjaga interpretasi tetap grounded.*

Module 07 menjawab "aset apa yang mungkin ada?". Module 08 menjawab "host mana yang reachable, port apa yang terlihat, service apa yang berjalan, dan apa bukti manualnya?".

Discovery → Hipotesis → Validasi (tool) → Verifikasi (manual) → Evidence → Hipotesis berikutnya

Tool (Nmap) membantu mempersempit kemungkinan dengan cepat; **manual verification** menjaga interpretasi tetap grounded pada protocol response yang sebenarnya, bukan asumsi dari signature database.

### 3. Host Discovery

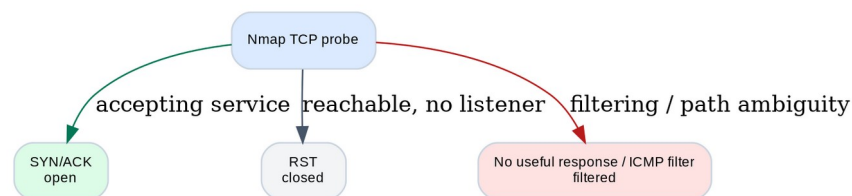
Sebelum memindai ribuan port, kita persempit dulu ke host yang hidup/menarik. Nmap melakukan host discovery secara default sebelum port scan.

| Command                      | Makna                                                                         |
|------------------------------|-------------------------------------------------------------------------------|
| <code>nmap -sn TARGET</code> | Host discovery saja, tanpa port scan                                          |
| <code>nmap -sL TARGET</code> | List scan — daftar target tanpa port scan, tapi bisa melakukan DNS resolution |
| <code>nmap -Pn TARGET</code> | Lewati discovery, perlakukan semua target seolah online                       |
| <code>nmap -n ...</code>     | Lewati reverse DNS untuk mengurangi noise/latency                             |

#### Common mistake.

-Pn **bukan** "stealth mode". Ia cuma melewati host-discovery phase, dan justru bisa membuat scan lebih lambat karena semua target diperlakukan online (semua port di-probe meski host sebenarnya mati).

### 4. Port States: Enam, Bukan Dua



*Simplified intuition for common TCP outcomes.*

Nmap tidak cuma mengenal open/closed — ada enam state, dan semuanya menggambarkan **apa yang dilihat scanner dari lokasi & teknik tertentu**, bukan sifat permanen sebuah port.

| State           | Interpretasi praktis                                                                 |
|-----------------|--------------------------------------------------------------------------------------|
| open            | Ada aplikasi yang menerima koneksi/datagram                                          |
| closed          | Host reachable, tapi tidak ada listener di port itu                                  |
| filtered        | Filter/network path mencegah Nmap menentukan open vs closed                          |
| unfiltered      | Port reachable, tapi teknik scan tertentu belum menentukan open/closed               |
| open filtered   | Tidak cukup bukti untuk membedakan open dan filtered — umum pada UDP                 |
| closed filtered | Tidak cukup bukti untuk membedakan closed dan filtered — muncul pada teknik tertentu |

### 5. TCP Connect vs SYN Scan

| Scan        | Contoh                             | Kapan berguna                     |
|-------------|------------------------------------|-----------------------------------|
| TCP Connect | <code>nmap -sT -p 80 TARGET</code> | Tidak butuh raw-packet privilege; |

|          |                                         |                                                  |
|----------|-----------------------------------------|--------------------------------------------------|
|          |                                         | pakai connect ( ) OS secara penuh                |
| SYN Scan | <code>sudo nmap -sS -p 80 TARGET</code> | Raw SYN probe; efisien, butuh privilege elevated |

Perbedaannya bukan soal "mana yang lebih stealth" — fokus ke mekanisme (full handshake vs half-open) dan telemetry apa yang dihasilkan tiap teknik di sisi defender.

## 6. Memilih Port dengan Sengaja

Default Nmap memindai 1.000 port TCP paling umum. Untuk lab atau engagement, pilih port berdasarkan **tujuan**, bukan karena flag terlihat keren.

| Pilihan        | Contoh                       | Catatan                                                                                               |
|----------------|------------------------------|-------------------------------------------------------------------------------------------------------|
| Specific ports | <code>-p 22, 80, 443</code>  | Cepat dan hypothesis-driven                                                                           |
| Range          | <code>-p 1-1024</code>       | Cakupan eksplisit                                                                                     |
| Top ports      | <code>--top-ports 100</code> | Trade-off kecepatan vs coverage                                                                       |
| All TCP ports  | <code>-p -</code>            | 1–65535; mahal, hanya kalau scope & waktu mendukung                                                   |
| Open only      | <code>--open</code>          | Menyederhanakan output — tapi jangan sampai kehilangan evidence state lain kalau audit membutuhkannya |

### Think like an attacker.

Kalau recon Module 07 sudah menunjukkan web app di custom port, scan "top 1000" saja bisa gagal menemukan service penting. Pakai bukti recon untuk menentukan coverage — bukan default tool begitu saja.

## 7. Service & Version Detection ( -sv)

Nomor port cuma petunjuk — service bisa berjalan di port yang tidak biasa. -sv mengirim probe dan mencocokkan response dengan signature database untuk mengidentifikasi protocol, product, version, dan kadang CPE.

```
nmap -sv -p 80,6379 172.31.80.10
nmap -sv --version-light -p 80 172.31.80.10
```

### Scanner output is evidence, not verdict.

Banner/version detection bisa salah, diubah, diproxy, atau sengaja disembunyikan. Selalu cari konfirmasi manual (§11) sebelum menarik kesimpulan vulnerability.

## 8. UDP Enumeration

UDP tidak punya handshake seperti TCP. Banyak service UDP tidak merespons probe yang salah, firewall bisa membuang packet diam-diam, dan ICMP error sering dibatasi — makanya UDP scan umumnya lebih lambat dan sering menghasilkan open|filtered.

```
sudo nmap -sU -p 9999 172.31.80.13
sudo nmap -sU -sv -p 9999 172.31.80.13
```

### Protocol matters.

DNS, SNMP, DHCP, NTP, dan banyak service penting lain memakai UDP. Mengabaikan UDP sepenuhnya bisa membuat inventory lo tidak lengkap.

## 9. Nmap Scripting Engine (NSE)

NSE menjalankan script Lua untuk discovery, version enrichment, authentication checks, vulnerability checks, bahkan kategori intrusive/exploit. **Script tidak berjalan dalam sandbox** — selalu baca dokumentasinya dulu sebelum eksekusi.

```
nmap --script-help http-title
nmap --script-help redis-info

nmap -p 80 --script http-title,http-headers 172.31.80.10
nmap -p 6379 --script redis-info 172.31.80.12
```

### Peringatan `-sC`.

`-sC` setara dengan `--script=default`, dan dokumentasi Nmap sendiri menyatakan sebagian default script dianggap intrusive. Di module ini, `-sC` hanya boleh dicoba pada lab lokal milik sendiri. Untuk engagement nyata, pilih script **eksplisit** berdasarkan RoE — jangan pernah `--script all/vuln/exploit/dos/brute` tanpa izin tertulis yang menyebutkan dampaknya.

## 10. Output & Evidence

Jangan cuma copy-paste terminal. `-oA` menyimpan beberapa format sekaligus:

```
mkdir -p module08-enum/02-scans
nmap -sV -p 80 -oA module08-enum/02-scans/nginx 172.31.80.10
```

| Ekstensi            | Isi                                                                      |
|---------------------|--------------------------------------------------------------------------|
| <code>.nmap</code>  | Human-readable normal output                                             |
| <code>.gnmap</code> | Legacy grepable output — praktis tapi tidak sekaya XML                   |
| <code>.xml</code>   | Paling cocok untuk parser, inventory, dan tooling berikutnya (lihat §16) |

### Evidence integrity.

Simpan command, timestamp, target, dan hasil mentah. Screenshot bagus untuk report, tapi raw output jauh lebih mudah diaudit dan di-parse ulang.

## 11. Timing & Noise

Nmap punya timing template `-T0` sampai `-T5` plus banyak kontrol rate/retry. **Semakin agresif bukan berarti semakin benar** — pada jaringan lambat, packet loss justru bisa membuat hasil lebih buruk (false negative).

- Default timing adalah baseline yang baik untuk banyak kasus.
- Jangan otomatis pakai `-T5` pada production network.
- Parallelism/rate tinggi bisa memicu alert defender atau membebani target.
- Catat setiap perubahan timing di activity log — ini memengaruhi repeatability hasil.

## 12. Protocol-specific Enumeration Mindset

Setelah service teridentifikasi, pertanyaan berubah dari *"port apa yang open?"* menjadi *"apa yang bisa dipelajari dari protocol ini tanpa melampaui scope?"*

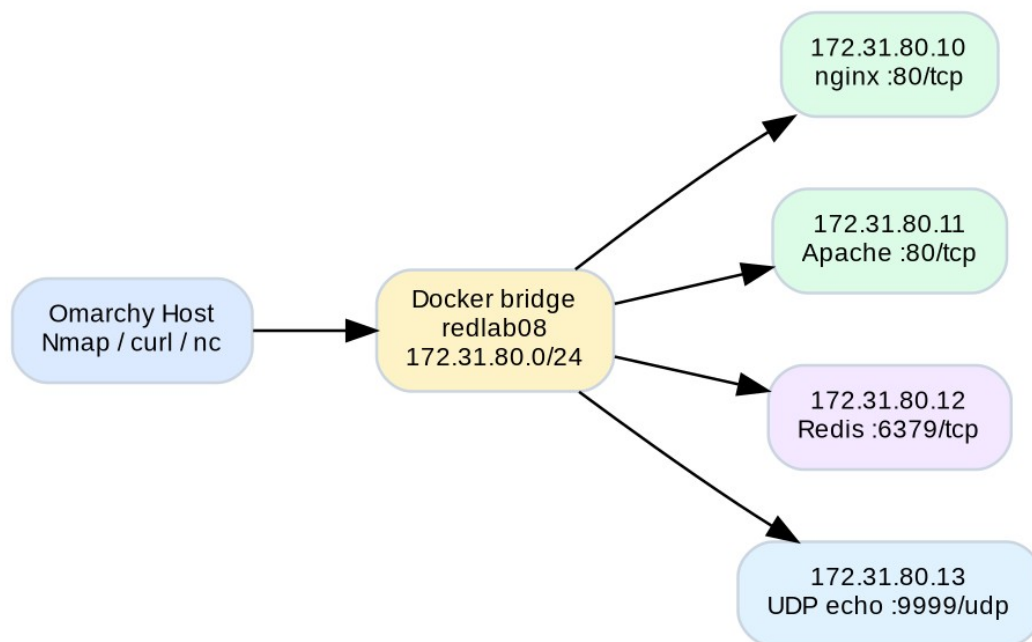
| Service    | Pertanyaan awal                                      | Verifikasi aman                                                      |
|------------|------------------------------------------------------|----------------------------------------------------------------------|
| HTTP/HTTPS | Status, headers, title, redirect, TLS, virtual host? | <code>curl -I / curl -v</code> ; browser/Burp pada target authorized |
| SSH        | Banner, protocol, host key, auth                     | Version detection saja; jangan brute                                 |

|       |                                                         |                                                           |
|-------|---------------------------------------------------------|-----------------------------------------------------------|
|       | methods?                                                | force tanpa izin                                          |
| FTP   | Banner, anonymous login diizinkan?                      | Explicit check hanya kalau RoE mengizinkan                |
| SMB   | Dialect, signing, shares, domain context?               | Diperdalam di Module 18/19                                |
| DNS   | Authoritative records, recursion, zone-transfer policy? | dig; zone transfer hanya kalau eksplisit diizinkan        |
| SNMP  | Version/community configuration?                        | UDP enumeration; credential guessing perlu izin eksplisit |
| LDAP  | Directory endpoint, TLS, anonymous bind policy?         | Diperdalam di module AD                                   |
| Redis | Protocol reachable, authentication/protected mode?      | PING/INFO lewat RESP, hanya pada lab/authorized target    |

#### Stop condition.

Kalau sebuah teknik mulai mengubah data, menebak credential, membuat banyak request, atau berpotensi mengganggu availability — kembali ke RoE. Enumeration tidak berarti semua probe otomatis boleh.

## 13. Lab Topology



*Four intentionally simple services on a private Docker bridge.*

Docker bridge: redlab08 (subnet 172.31.80.0/24, tanpa -p/--publish ke host)

```

172.31.80.10  nginx:alpine      → HTTP
172.31.80.11  httpd:alpine   → HTTP (Apache)
172.31.80.12  redis:alpine    → Redis (--protected-mode no, training only)
172.31.80.13  python:3-alpine → UDP echo custom (udp-echo.py, port 9999)
  
```

Tidak ada container yang dipublish ke interface host, jadi service tidak sengaja terbuka ke LAN. Host Omarchy tetap bisa mengakses bridge IP tersebut langsung lewat Docker networking standar.

```
cd modules/08-network-scanning-enumeration/labs
./multi-service-lab.sh create
./multi-service-lab.sh status
```

Cleanup setelah selesai:

```
./multi-service-lab.sh cleanup
```

## 14. Hands-on Lab 1 — Workspace & Scope Guardrail

```
./lab-init.sh module08-enum
cat module08-enum/00-scope/README.md
```

lab-nmap.py adalah **safety wrapper** yang hanya menerima target di 127.0.0.0/8 atau 172.31.80.0/24. Guardrail ini **bukan pengganti authorization** — tujuannya cuma mencegah typo target selama training.

```
# Expected: ditolak
python lab-nmap.py 192.168.1.1 -- -sn

# Allowed training subnet
python lab-nmap.py 172.31.80.0/28 -- -sn -n
```

## 15. Hands-on Lab 2 — Host Discovery

```
./multi-service-lab.sh create
python lab-nmap.py 172.31.80.0/28 -- -sn -n -oA module08-enum/01-discovery/hosts
```

Subnet /28 meliputi .0–.15, jadi gateway Docker dan container .10–.13 seharusnya muncul.

[ ] Temukan minimal empat container training.

[ ] Bedakan gateway bridge dari service container.

[ ] Simpan output normal **dan** XML.

## 16. Hands-on Lab 3 — Open vs Closed TCP

```
python lab-nmap.py 172.31.80.10 -- -n -p 80,65000
```

Port 80 harus open (nginx). Port 65000 biasanya c losed. Kalau hasilnya beda, troubleshoot pakai docker ps/ss/docker inspect dulu — jangan langsung ubah scan flags.

## 17. Hands-on Lab 4 — SYN vs Connect Scan

```
nmap -sT -n -p 80 172.31.80.10
sudo nmap -sS -n -p 80 172.31.80.10
```

Bandingkan output, kebutuhan privilege, dan packet behavior-nya. Jangan mengejar istilah "stealth" — fokus ke mekanisme dan telemetry yang dihasilkan.

## 18. Hands-on Lab 5 — Service/Version Detection

```
nmap -n -sV -p 80 172.31.80.10
nmap -n -sV -p 80 172.31.80.11
nmap -n -sV -p 6379 172.31.80.12
```

| Target          | Expected family | Manual follow-up                |
|-----------------|-----------------|---------------------------------|
| 172.31.80.10:80 | nginx           | curl -I<br>http://172.31.80.10/ |
| 172.31.80.11:80 | Apache httpd    | curl -I<br>http://172.31.80.11/ |

172.31.80.12:6379

Redis

RESP PING via nc

Tujuan exercise ini: membuktikan dua host di port 80 yang sama bisa menjalankan product yang berbeda — nomor port bukan satu-satunya sumber identifikasi.

## 19. Hands-on Lab 6 — Manual Verification

```
./manual-verify.sh
```

Script ini menjalankan `curl -I` ke kedua web server, RESP PING manual ke Redis (`*1\r\n$4\r\nPING\r\n`), dan UDP probe manual ke echo service. Bandingkan hasilnya dengan output `-sv` — kalau scanner menampilkan version tapi response manual tidak mendukung detail itu, catat sebagai **confidence rendah**, bukan asumsi pasti.

## 20. Hands-on Lab 7 — UDP

```
printf 'hello' | nc -u -w 2 172.31.80.13 9999
sudo nmap -sU -n -p 9999 172.31.80.13
```

UDP echo service akan membalas dengan marker `REDLAB08 UDP ECHO: hello`. Manual packet exchange ini memberi bukti bahwa service **memang menerima datagram** — bukan cuma inference open | filtered dari Nmap.

## 21. Hands-on Lab 8 — Explicit NSE

```
nmap --script-help http-title
nmap -n -p 80 --script http-title,http-headers 172.31.80.10
nmap -n -p 6379 --script redis-info 172.31.80.12
```

### Read before run.

Selalu `--script-help` dulu untuk script baru. Hindari kategori `all`, `vuln`, `exploit`, `dos`, `brute`, atau `intrusive` kecuali lab/engagement memang membutuhkannya secara eksplisit dan mengizinkan dampaknya.

## 22. Hands-on Lab 9 — Structured Evidence

```
nmap -n -sv -p 80,6379 -oA module08-enum/02-scans/selected 172.31.80.10 172.31.80.12

python nmap-inventory.py \
  module08-enum/02-scans/selected.xml \
  --output module08-enum/04-inventory/services.csv

column -s, -t < module08-enum/04-inventory/services.csv | less -S
```

`nmap-inventory.py` mem-parse XML (host, host\_state, protocol, port, port\_state, service, product, version, extrainfo) jadi CSV ternormalisasi. **File XML tetap dipertahankan sebagai evidence sumber** — jangan ganti evidence mentah dengan hasil transformasi.

## 23. Hands-on Lab 10 — Hypothesis-driven Enumeration (No Walkthrough)

Tanpa walkthrough, jalankan proses berikut pada keempat target lab dan tulis **alasan** tiap command di notes:

- Host discovery pada /28.
- Pilih port scan yang masuk akal untuk setiap host.
- Jalankan `-sv` hanya pada port yang relevan.
- Pilih maksimal dua NSE script eksplisit untuk HTTP/Redis.
- Lakukan manual verification.
- Update inventory CSV dan buat summary singkat.

**Success criterion.**

Lo bisa menjelaskan dari mana setiap kesimpulan berasal: packet/scan output, service probe, NSE, atau manual protocol response — bukan "karena Nmap bilang begitu".

## 24. Mini Project — Enumeration Case File

```
module08-enum/
├── 00-scope/
├── 01-discovery/
├── 02-scans/
├── 03-manual/
├── 04-inventory/services.csv
├── 05-evidence/
└── 06-summary/enumeration-summary.md
```

Summary minimal harus mencantumkan: target, open ports, service/product/version + confidence-nya, manual evidence, unknowns, dan next safe question.

## 25. Practical Challenge — 45 Menit

Mulai dari lab yang baru dibuat. Jangan lihat catatan section sebelumnya. Target: buat inventory yang bisa direview orang lain.

- [ ] Buat workspace baru.
- [ ] Discover host pada 172.31.80.0/28.
- [ ] Identifikasi service keempat container.
- [ ] Gunakan minimal satu TCP SYN scan, satu TCP connect scan, dan satu UDP scan.
- [ ] Gunakan -sV pada port terpilih.
- [ ] Gunakan explicit NSE hanya setelah baca --script-help.
- [ ] Manual verify semua service.
- [ ] Export -oA dan parse XML ke CSV.
- [ ] Tulis 5–10 baris summary tanpa mengklaim vulnerability yang belum dibuktikan.
- [ ] Cleanup Docker lab.

**Tidak lulus kalau...**

Lo cuma menjalankan nmap -A lalu menyalin output. Goal module ini adalah methodology dan reasoning, bukan satu command serba bisa.

## 26. Common Mistakes

| Kesalahan                        | Kenapa bermasalah                         | Perbaikan                              |
|----------------------------------|-------------------------------------------|----------------------------------------|
| Scanning tanpa scope             | Risiko legal/operasional                  | Validasi target list dan RoE dulu      |
| Menganggap port number = service | Service bisa pindah port                  | -sV + manual verification              |
| Langsung -A/-sC ke production    | Menambah probe/script tanpa review        | Pilih fitur eksplisit sesuai tujuan    |
| Mengabaikan UDP                  | Inventory bisa kehilangan service penting | UDP scan hypothesis-driven             |
| Tidak menyimpan XML              | Sulit diotomasi dan diaudit               | Selalu pakai -oA                       |
| Percaya banner mentah            | Banner bisa spoofed/disembunyikan         | Cross-check dengan manual verification |

|                                |                             |                                      |
|--------------------------------|-----------------------------|--------------------------------------|
| Terlalu cepat menaikkan timing | False negative / disruption | Mulai konservatif, ukur, baru naikan |
|--------------------------------|-----------------------------|--------------------------------------|

## 27. Knowledge Check

- Kenapa state port Nmap bukan properti permanen?
- Apa beda -sn dan -Pn?
- Kapan -sT lebih berguna dibanding -sS?
- Kenapa service detection tidak cukup hanya dari nomor port?
- Apa arti open|filtered pada UDP?
- Kenapa -sC perlu diperlakukan hati-hati?
- Apa keuntungan -oA dan format XML dibanding .nmap/.gnmap?
- Kenapa manual verification tetap wajib setelah -sV?
- Kenapa -T5 bukan default yang baik untuk semua kasus?
- Apa beda discovery, enumeration, dan vulnerability validation?

## 28. Exit Criteria

- [ ] Bisa menjelaskan enam Nmap port states.
- [ ] Bisa menjalankan host discovery dan port scan pada lab authorized.
- [ ] Paham perbedaan -sT, -sS, dan -sU.
- [ ] Bisa pakai -sV tanpa memperlakukan output sebagai kebenaran absolut.
- [ ] Bisa membaca --script-help dan memilih NSE script secara eksplisit.
- [ ] Bisa menyimpan -oA dan parse XML ke CSV.
- [ ] Bisa manual-verify HTTP, Redis, dan UDP echo di lab.
- [ ] Memahami bahwa scanning tetap harus mengikuti scope dan RoE.

## 29. Deliverables

- docx/Red-Team-Module-08-Network-Scanning-Enumeration-0marchy.docx — versi editable
- pdf/Red-Team-Module-08-Network-Scanning-Enumeration-0marchy.pdf — versi final
- labs/:
- lab-init.sh — membuat workspace enumeration (00-scope ... 06-summary)
- multi-service-lab.sh — membuat/menghapus isolated Docker lab (nginx, Apache, Redis, UDP echo)
- udp-echo.py — UDP echo service custom untuk training
- lab-nmap.py — Nmap safety wrapper, hanya menerima 127.0.0.0/8 + 172.31.80.0/24
- nmap-inventory.py — parse Nmap XML menjadi CSV inventory
- manual-verify.sh — manual protocol verification (HTTP, Redis, UDP)

## 30. Cheat Sheet

```
# Host discovery
nmap -sn -n 172.31.80.0/28

# TCP connect / SYN
nmap -sT -n -p 80 172.31.80.10
sudo nmap -sS -n -p 80 172.31.80.10

# Service detection
nmap -sV -n -p 80,6379 TARGET

# UDP
sudo nmap -sU -n -p 9999 172.31.80.13

# Explicit NSE
nmap --script-help http-title
nmap -p 80 --script http-title,http-headers TARGET

# Evidence
nmap -sV -oA scan-name TARGET
python labs/nmap-inventory.py scan-name.xml --output services.csv

# Cleanup
./labs/multi-service-lab.sh cleanup
```

## 31. References

- Nmap — Port Scanning Basics — <https://nmap.org/book/man-port-scanning-basics.html>
- Nmap — Host Discovery — <https://nmap.org/book/man-host-discovery.html>
- Nmap — Service and Version Detection — <https://nmap.org/book/man-version-detection.html>
- Nmap — UDP Scan — <https://nmap.org/book/scan-methods-udp-scan.html>
- Nmap — Scripting Engine (NSE) — <https://nmap.org/book/man-nse.html>
- Arch Linux — nmap package — [https://archlinux.org/packages/extra/x86\\_64/nmap/](https://archlinux.org/packages/extra/x86_64/nmap/)
- Docker — Bridge network driver — <https://docs.docker.com/engine/network/drivers/bridge/>

### Next module:

09-vulnerability-assessment — CVE, CWE, CVSS, scanner output, false positives, version correlation, reachability, manual validation, prioritization, dan remediation.